# Process Book: Formula 1 Performance Trend Dashboard

**Author:** Weihao (Oliver) Song
**Course:** CS-6630 Visualization for Data Science
**Date:** Fall 2025

---

## 1. Overview and Motivation

Formula 1 is a sport defined by data. In a single race, a car transmits gigabytes of telemetry. However, for the average fan, this data is often summarized into a single metric: **The Final Position**.

The goal of this project is to move beyond the leaderboard and visualize the **Lap Time Evolution** of drivers. By plotting every single lap, rather than just an average, we can reveal patterns of consistency, tire degradation, and strategy that are invisible in standard race summaries.

---

## 2. Related Work

Most existing F1 visualizations (like the TV broadcast) focus on "Gap to Leader" (line charts).

- **Existing approach:** Line charts showing time gaps relative to the first-place car.
- **My approach:** A scatter plot of **absolute lap times**.

I chose this approach because "Gap to Leader" hides raw performance. If the leader slows down, the gap stays the same, even if the driver behind is also driving poorly. My visualization isolates the driver's individual performance against the track.

---

## 3. Guiding Questions

1. **Consistency:** Does a driver have a high variance (erratic) or low variance (robot-like)?
2. **Track Comparison:** How do lap time profiles differ between a short, technical track (Monaco) and a high-speed track (Las Vegas)?
3. **Driver Style:** Can we quantify abstract concepts like "Aggression" or "Smoothness" using raw lap data?

---

## 4. Data Pipeline

### 4.1 Data Source

I utilized the **FastF1** Python library to access the official F1 Live Timing API.

### 4.2 Processing Challenges

- **Time Alignment:** Lap times and Weather data had different timestamps. I used `pandas.merge_asof` to align the nearest weather data point to the start of every lap.
- **Outliers:** Safety Car laps (120s+) ruined the chart scale. I implemented a filter to remove laps slower than 107% of the racing pace.
- **Multi-Race Support:** The pipeline was designed to merge multiple races (e.g., Monaco + Vegas) into a single `laps_cleaned.csv` with a `source_file` column for filtering.

---

## 5. Visualization Design Evolution

### Phase 1: The Raw Prototype

Initially, I plotted data using Python's Matplotlib.

- *Critique:* It was static. I couldn't hover to see tire data, and switching drivers required rewriting code.

### Phase 2: Moving to D3.js

I migrated the frontend to D3.js to enable web-based interactivity.

- *Challenge:* The axes had no labels. Users didn't know what "80" on the Y-axis meant.
- *Solution:* I implemented dynamic SVG text labels and adjusted margins to ensure "Lap Time (s)" was always visible.

### Phase 3: The "Sort by Speed" Feature

- **The Problem:** A chronological plot shows history, but it is visually difficult to assess pure consistency when laps fluctuate due to traffic.
- **The Solution:** I added a "Sort by Speed" toggle.
- **Result:** This transforms the scatter plot into a distribution curve (S-Curve). A flat horizontal line instantly signals a consistent driver, while a steep vertical curve indicates high degradation or inconsistency.

### Phase 4: The Profile Page & Hexagon Analysis

I realized the main dashboard was becoming too crowded to show detailed metrics.

- **Architecture Change:** I created a dedicated sub-page (`profile.html`) linked via a button in the sidebar.
- **The Radar Chart:** To summarize a driver's weekend in a single shape, I built a 5-axis Hexagon Chart.
- **The Metric Challenge:** Raw statistics (like Standard Deviation) are hard for users to interpret.
    - *Fix:* I converted all stats into a **0–100 Score**.
    - *Example:* Instead of displaying "Std Dev: 1.2s", I display "Smoothness: 82/100" using the formula $100 - (15 \times \sigma)$.

---

## 6. Implementation Details

**Technology Stack:**

- **Python (Pandas, FastF1):** For ETL (Extract, Transform, Load).
- **HTML/CSS:** For the dashboard layout and "Dark Mode" styling (mimicking the F1 TV aesthetic).
- **JavaScript (D3 v7):** For binding data to the DOM and managing updates.

**Key Algorithm: Robust Scoring System** To prevent the dashboard from crashing on slow drivers (where math might yield `Infinity` or `NaN`), I implemented a clamping system in `profile.js`:

```
// Ensure score is always 0-100
const clamp = (n) => Math.max(0, Math.min(100, n || 0));

// Limit Pushing: Ratio of Personal Best to Average
// If Avg is close to Best, score is high.
let pushScore = clamp((myBest / myAvg) * 100);

// Smoothness: Linear decay based on Standard Deviation
let smoothScore = clamp(100 - (myStd * 15));
```